Diya Nagdev
D15A - 04
ML-DL Experiment 6

Apply K-Means and Hierarchical Clustering on sample datasets

1. Dataset Source

The dataset used for this experiment is the **Fake Social Media Account Detection**

Dataset, obtained from Kaggle.

Source Link:
https://www.kaggle.com/datasets/free4ever1/instagram-fake-spammer-genuine-accounts?resource=download

This dataset contains information about Instagram profiles used to detect whether an account is **fake or genuine**.

2. Dataset Description

The **Instagram Fake Account Detection Dataset** contains various attributes related to user profiles such as account activity, follower statistics, and profile characteristics.

Dataset Characteristics:

Number of Instances: ~576 accounts

Features include:

- profile_pic – Whether the account has a profile picture
- nums_length_username – Ratio of numbers in username
- fullname_words – Words in full name
- external_URL – Presence of external URL
- private – Whether account is private

Target Variable:

fake
0 → Genuine Account
1 → Fake Account

Data Type: Numerical features

Nature of Dataset: Structured real-world social media profile data.

This dataset can be used to analyze social media behaviour and detect suspicious accounts.

3. Mathematical Formulation of Algorithms

K-Means Clustering

K-Means aims to minimize within-cluster variance.

Objective Function:

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||^2$$

Where:

- kkk = number of clusters
- CiC_iCi = cluster
- μi\mu_iμi = centroid

Steps:

1. Initialize k centroids
2. Assign each point to nearest centroid
3. Update centroids
4. Repeat until convergence

Hierarchical Clustering

Two types:

- **Agglomerative (bottom-up)**
- **Divisive (top-down)**

Distance Calculation:

Common metrics:

- Euclidean distance:

$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

Linkage Methods:

- Single linkage
- Complete linkage
- Average linkage

4. Algorithm Limitations

K-Means:

- Requires **predefined k**
- Sensitive to **initial centroids**
- Works poorly with:
  - Non-spherical clusters
  - Outliers

Hierarchical Clustering:

- Computationally expensive ($O(n^2)$)
- Difficult for large datasets
- No clear stopping rule

5. Methodology / Workflow

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load Dataset

```
import pandas as pd

df = pd.read_csv('/content/train.csv')
df.head()
```

	profile pic	nums/length username	fullname words	nums/length fullname	name==username	description length	external URL	private	#posts	#followers	#follows	fake
0	1	0.27	0	0.0	0	53	0	0	32	1000	955	0
1	1	0.00	2	0.0	0	44	0	0	286	2740	533	0
2	1	0.10	2	0.0	0	0	0	1	13	159	98	0
3	1	0.00	1	0.0	0	82	0	0	679	414	651	0
4	1	0.00	2	0.0	0	0	0	1	6	151	126	0

Step 3: Data Preprocessing

```
X = df.drop('fake', axis=1)
```

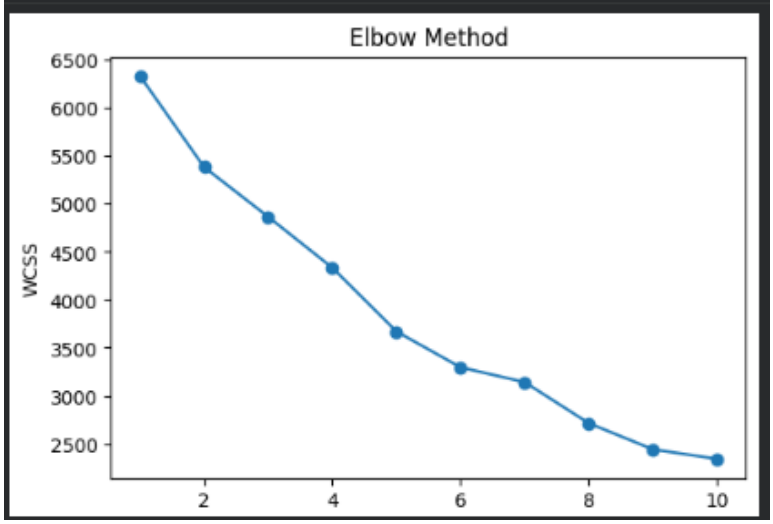
Step 4: K-Means Clustering

Find optimal K

```
wcss = []

for i in range(1,11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(6,4))
plt.plot(range(1,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()
```



Apply K-Means

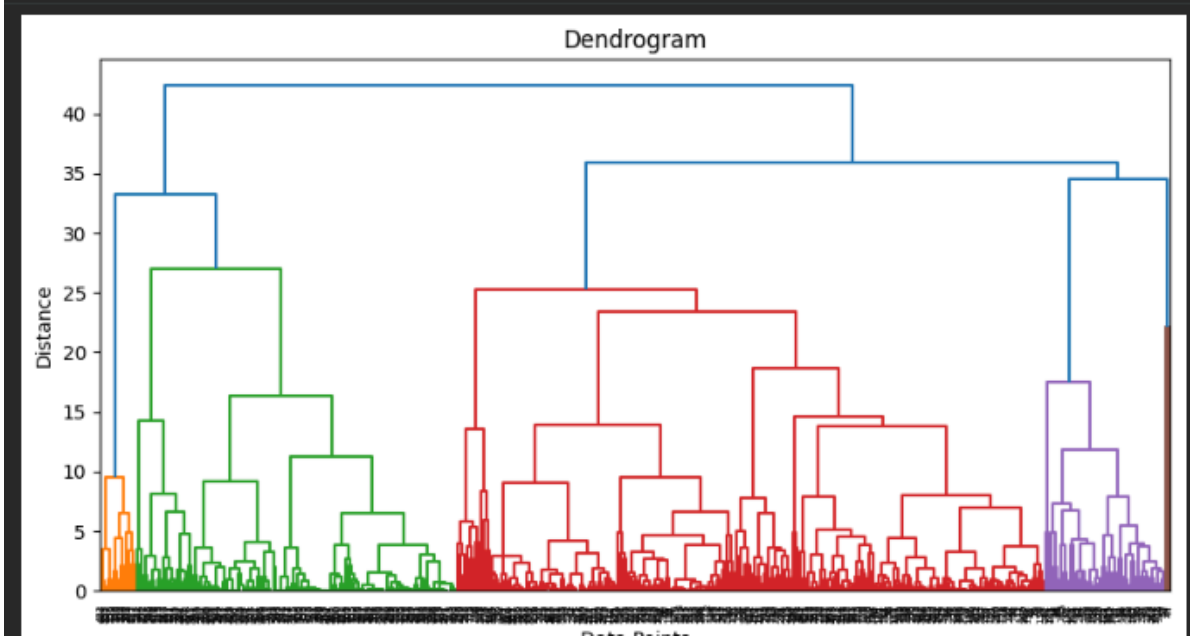
```
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)
```

Visualization

```
plt.figure(figsize=(6,4))
plt.scatter(X_pca[:,0], X_pca[:,1], c=kmeans_labels)
plt.title("K-Means Clustering (PCA)")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

Step 5: Hierarchical Clustering Dendrogram

```
plt.figure(figsize=(10,5))
dendrogram(linked)
plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
```

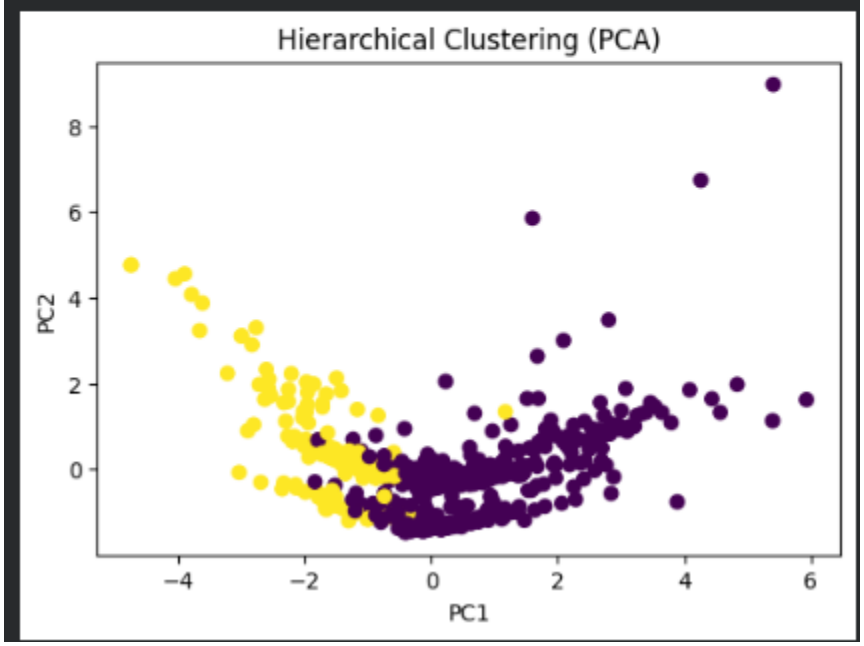


Apply Model

```
hc = AgglomerativeClustering(n_clusters=2, linkage='ward')
hc_labels = hc.fit_predict(X_scaled)
```

Visualization

```
plt.figure(figsize=(6,4))
plt.scatter(X_pca[:,0], X_pca[:,1], c=hc_labels)
plt.title("Hierarchical Clustering (PCA)")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```



6. Performance Analysis

Metrics:

Silhouette Score

```
kmeans_score = silhouette_score(X_scaled, kmeans_labels)
hc_score = silhouette_score(X_scaled, hc_labels)

print("KMeans Silhouette Score:", kmeans_score)
print("Hierarchical Silhouette Score:", hc_score)
```

```
KMeans Silhouette Score: 0.19174850490885298
Hierarchical Silhouette Score: 0.20914602307679223
```

Interpretation:

- Score ranges: **-1 to 1**
- Closer to **1 → better clustering**

7. Hyperparameter Tuning

K-Means:

- n_clusters → use Elbow Method
- init → random or k-means++
- max_iter

Hierarchical:

- n_clusters
- linkage:
  - ward
  - complete
  - average

8. Conclusion

- K-Means is:
  - Fast
  - Efficient for large datasets
  - Requires predefined clusters
- Hierarchical Clustering:
  - No need to define k initially
  - Provides dendrogram (visual insight)
  - Slower for large datasets

Final Insight:

Both methods successfully segment customers based on:

- Income
- Spending behavior

K-Means is preferred for scalability, while Hierarchical is useful for **understanding cluster structure**.